Myopic! 3-D Studio

Project Specification & Development Roadmap

TL;DR

A prompt-driven 3D storyboarding environment, built in Claude Code, that parses scene descriptions into editable components (lighting, camera, characters, props) and is architected to support Daz 3D asset libraries.

---

1. Vision Statement

Myopic! 3-D Studio is a storyboarding tool that lets a director describe a scene in natural language (typed or dictated) and immediately receive an editable, structured 3D scene layout — without manually configuring every element from scratch. The system interprets prompts and populates a scene graph with discrete, adjustable parameters for each component.

---

2. Core Workflow

```
[Prompt Input] → [Scene Parser / AI Interpreter] → [Scene Graph] → [3D Viewport +
                                                                        ↓
                                                                [Storyboard Export
```

Step-by-Step

1. **Input** — User types or dictates a scene description.
2. **Parse** — AI extracts scene components: setting, mood, characters, props, camera framing, lighting intent.
3. **Populate** — Each extracted element is mapped to an editable scene object.
4. **Edit** — User adjusts individual parameters via panel UI (no re-prompting required for tweaks).

5. **Export** — Scene exported as storyboard frame, shot list entry, or asset reference sheet.

---

3. Scene Components & Editable Parameters

3.1 Environment / Set

Parameter	Type	Notes
Location name	String	e.g., “Abandoned warehouse, exterior”
Time of day	Enum	Dawn / Morning / Midday / Dusk / Night
Weather / atmosphere	String	Fog, rain, clear, overcast
Background asset	File path	Daz environment scene or HDRI

3.2 Lighting

Parameter	Type	Notes
Lighting scheme	Enum	Natural / Studio / Dramatic / Practical
Key light direction	Vector / Slider	Azimuth + elevation
Key light color	Color picker	Hex or Kelvin temp
Fill light intensity	Float 0–1	Ratio to key
Rim / back light	Toggle + intensity	
Shadow softness	Float 0–1	
Mood preset	Enum	Noir / Golden hour / Overcast / Neon night, etc.

3.3 Camera

Parameter	Type	Notes
Shot type	Enum	ECU / CU / MCU / MS / MLS / LS / ELS

Camera angle	Enum	Eye level / Low / High / Dutch / Bird's eye / Worm's eye
Lens focal length	Integer (mm)	18 / 35 / 50 / 85 / 135 / 200
Depth of field	Float	F-stop equivalent
Focus subject	Reference	Links to character or prop object
Camera position	XYZ	Editable in viewport
Movement / motion	Enum	Static / Pan / Tilt / Dolly / Crane / Handheld
Aspect ratio	Enum	16:9 / 2.39:1 / 4:3 / 1:1

3.4 Characters / Figures

Each character is an independent object with its own parameter block.

Parameter	Type	Notes
Figure ID	String	Internal reference name
Daz figure asset	File path	`.duf` / `.dsf` reference
Pose preset	File path or label	Daz pose file or custom
Facial expression	Enum / slider	Daz morph targets
Position in scene	XYZ	
Costume / outfit	File path	Daz wearable asset
Scale	Float	Relative to scene units
Visibility	Toggle	

3.5 Props

Parameter	Type	Notes

Prop ID	String	Internal reference name
Asset source	File path	Daz prop `.duf` or custom mesh
Position	XYZ	
Rotation	XYZ	
Scale	XYZ	Non-uniform scaling supported
Material override	Toggle + file path	Optional texture swap
Visibility	Toggle	

---

4. Prompt Parser — AI Interpretation Layer

The parser is the intelligence layer between user language and the scene graph.

4.1 Extraction Targets

When a prompt is submitted, the AI extracts and categorizes:

- **Who** — Named or described characters, count, relative positions
- **Where** — Setting, environment type, interior/exterior
- **When** — Time of day, era, season
- **What** — Props, objects of importance, story context
- **How it feels** — Mood, tone, genre conventions (used for lighting/camera defaults)
- **How it's framed** — Explicit or implied camera language ("close on her face," "wide establishing shot")

4.2 Ambiguity Handling

- If a parameter cannot be confidently inferred, the system flags it with a **[?]** indicator in the editor panel.
- User resolves flagged items manually before finalizing the frame.
- Re-prompting a partial scene updates only the flagged or selected components.

4.3 Prompt Examples

> *"Interior. A cramped server room, late night. Banks of blinking servers. A lone technician hunches over a terminal, face lit by screen glow. Tight over-the-shoulder shot."*

Expected extractions:

- **Environment:** Interior, server room, night
- **Atmosphere:** Artificial light, contained, tense
- **Character:** 1 figure, hunched pose, forward-facing
- **Lighting:** Practical (screen), low ambient, high contrast
- **Camera:** Over-the-shoulder, medium close-up, tight framing

---

5. Daz 3D Integration

5.1 Asset Library Connection

Daz 3D stores assets in a structured folder hierarchy. Myopic! will interface with this library by indexing the following:

Asset Type	Daz Default Path	File Types
Figures	`/My DAZ 3D Library/People/`	`.duf`, `.dsf`
Poses	`/My DAZ 3D Library/Poses/`	`.duf`
Props	`/My DAZ 3D Library/Props/`	`.duf`, `.obj`, `.fbx`
Environments	`/My DAZ 3D Library/Environments/`	`.duf`
Outfits / Wearables	`/My DAZ 3D Library/Clothing/`	`.duf`
Materials / Shaders	`/My DAZ 3D Library/Shader Presets/`	`.duf`, `.dsa`
Hair	`/My DAZ 3D Library/Hair/`	`.duf`
Lights	`/My DAZ 3D Library/Light Presets/`	`.duf`

> **Note:** Paths may vary if the user has configured custom content directories in Daz Studio. The integration layer should read from `DAZ Studio → Content Directory Manager` export or a user-defined config file.

5.2 Asset Index

- On first run, Myopic! scans configured Daz folders and builds a **local asset index** (JSON or SQLite).
- Index stores: asset name, type, relative path, thumbnail path (if available), tags (parsed from folder structure).
- Index is **refreshed on demand** or on detected file changes.

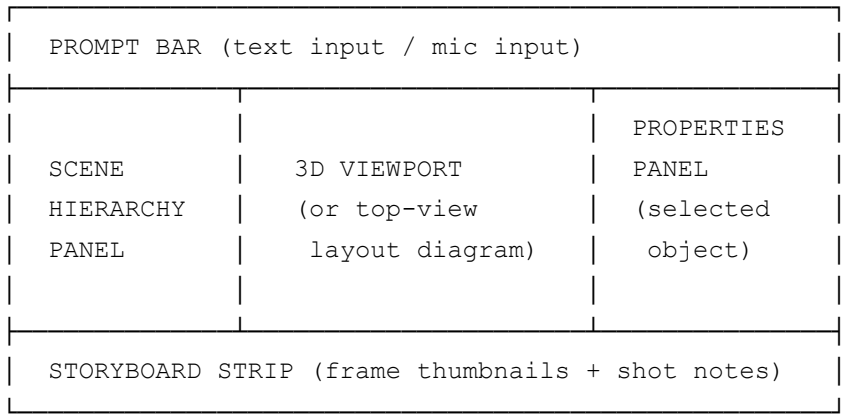
5.3 Integration Milestones

Phase	Goal
Phase 1	Manual asset path entry per scene component
Phase 2	Asset browser panel — searchable index of Daz library
Phase 3	AI-assisted asset suggestion (prompt keywords → matching Daz assets)
Phase 4	Scene export as Daz Studio `.duf` scene file or DazScript for auto-population

---

6. UI Architecture

6.1 Panel Layout (Reference)



6.2 Scene Hierarchy Panel

- Lists all objects in the scene as a tree: Environment → Lights → Camera → Characters → Props

- Clicking any item selects it and opens its parameter block in the Properties Panel
- Visibility toggles and lock controls per object

6.3 Properties Panel

- Dynamically populated based on selected object type
- All parameters are directly editable (sliders, dropdowns, text fields, color pickers)
- “AI Suggest” button on any field re-invokes the parser for that parameter only

6.4 Storyboard Strip

- Each scene state saved as a frame
- Frames are drag-reorderable
- Shot metadata per frame: shot number, duration estimate, director notes, camera label

---

7. Data Model

7.1 Scene File Format (`.myo` — proposed)

```
{
  "scene_id": "uuid",
  "title": "Scene 12 — Server Room",
  "created": "2026-04-28T00:00:00Z",
  "prompt": "Interior. A cramped server room...",
  "environment": { ... },
  "lighting": { ... },
  "camera": { ... },
  "characters": [ { ... }, { ... } ],
  "props": [ { ... } ],
  "storyboard_notes": "",
  "flagged_params": ["camera.focal_length"]
}
```

7.2 Asset Index Format

```
{
  "generated": "2026-04-28T00:00:00Z",
  "daz_root": "/Users/username/Documents/DAZ 3D/Studio/My Library",
  "assets": [
```

```
    {
      "id": "uuid",
      "name": "Genesis 9 Starter Essentials",
      "type": "figure",
      "path": "People/Genesis 9/...",
      "tags": ["figure", "genesis9", "base"]
    }
  ]
}
```

---

8. Development Roadmap

Phase 1 — Core Scaffold (Claude Code)

- Prompt input → structured JSON output (scene graph)
- Scene Hierarchy panel (static tree, no 3D render)
- Properties panel with editable fields per component type
- Save / load `.myo` scene file
- Basic storyboard strip (frame capture as JSON state)

Phase 2 — Visual Viewport

- Top-down 2D layout diagram (SVG or canvas) showing character/prop positions and camera cone
- Camera framing preview (aspect ratio overlay)
- Lighting direction indicator

Phase 3 — Daz 3D Integration (Phase 1–2 above)

- Config file for Daz library path
- Asset index builder
- Asset browser panel (search + filter)
- Asset path linking to scene objects

Phase 4 — Export & Polish

- PDF storyboard export (frames + shot notes)

- DazScript or `.duf` scene export
- Voice/dictation input
- Prompt history and version diffing

---

9. Technical Stack (Proposed)

Layer	Technology
Runtime	Claude Code (Node.js environment)
AI / Parse	Anthropic API (`claude-sonnet-4` via `/v1/messages`)
UI Framework	React + Tailwind CSS
Scene State	Zustand or Context API
File I/O	Node.js `fs` module (for `.myo` and asset index)
Asset Indexer	Node.js recursive directory walker + JSON/SQLite
3D Preview	Three.js (Phase 2, lightweight diagram — not full render)
Export	`jsPDF` or Puppeteer for PDF storyboards

---

10. Open Questions / Decisions Pending

1. **Viewport depth** — Is a 2D top-down diagram sufficient, or is a true 3D preview (Three.js scene) required from the start?
2. **Daz export format** — DazScript automation vs. `.duf` JSON construction vs. simply producing a human-readable "setup sheet" for manual Daz loading.
3. **Voice input** — Browser Web Speech API (artifact) vs. system-level dictation (Claude Code native).
4. **Asset thumbnail display** — Daz stores thumbnails as `.jpg` alongside assets; confirm this path is consistent across library versions.
5. **Multi-user / cloud sync** — Single-user local tool for now, or eventual sync layer?
